Communications numériques 1

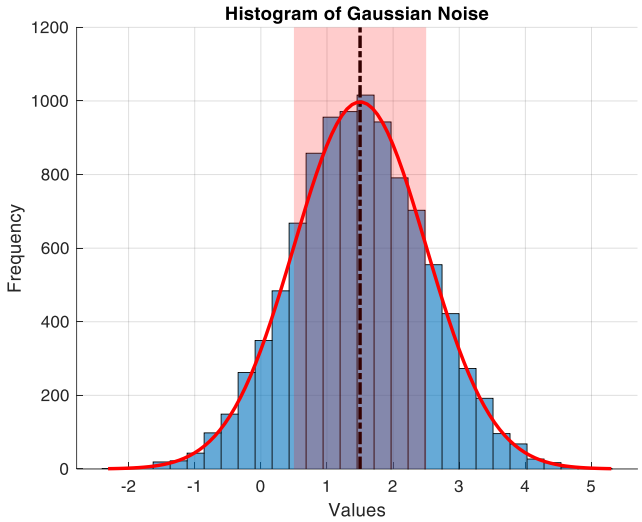
TDAO 2

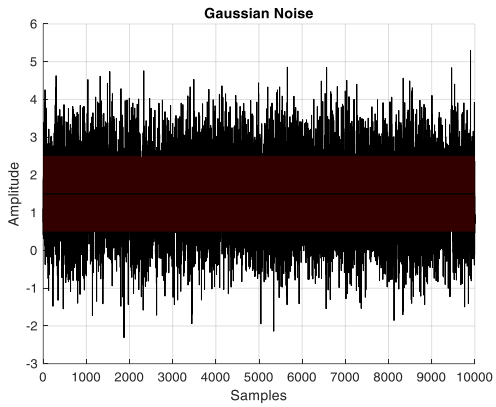
17/02/2025

A) Génération et analyse du bruit gaussien

Objectif : Comprendre la nature du bruit gaussien et visualiser sa distribution statistique et son spectre fréquentiel.

1. Générer un signal de bruit gaussien à 10000 échantillons avec une moyenne de 1,5 et un écart-type de 1
2. Tracer l'histogramme et comparer avec une distribution normale théorique





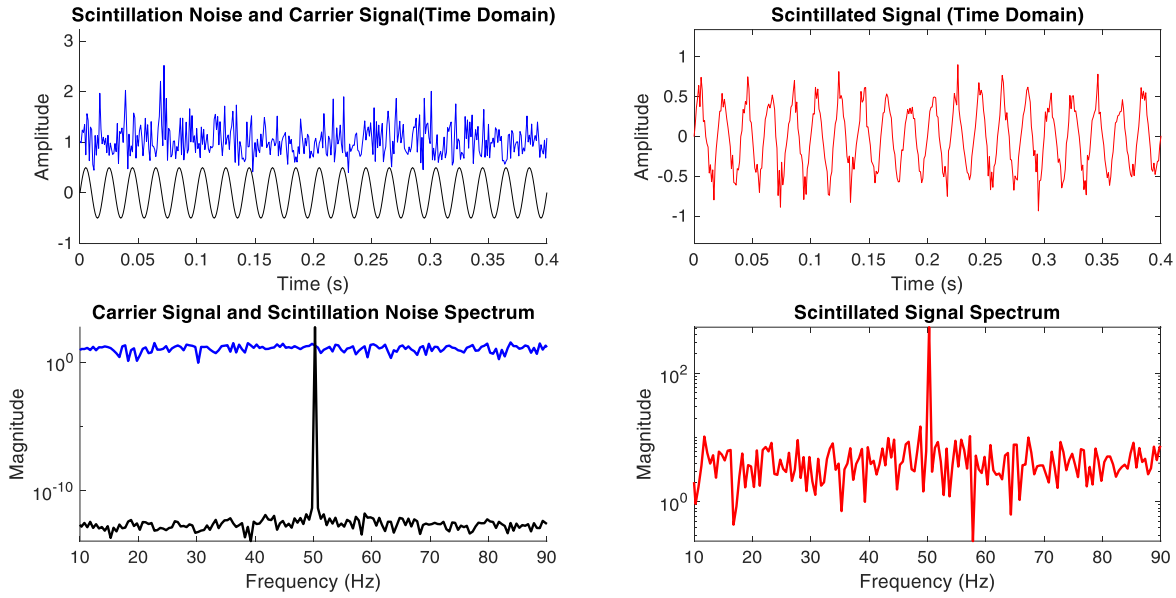

```
% Define parameters
N = 10000; % Number of samples
mu = 1.5; % Mean of the noise
sigma = 1; % Standard deviation of the noise

% fonctions utiles
% randn
% histogram
```

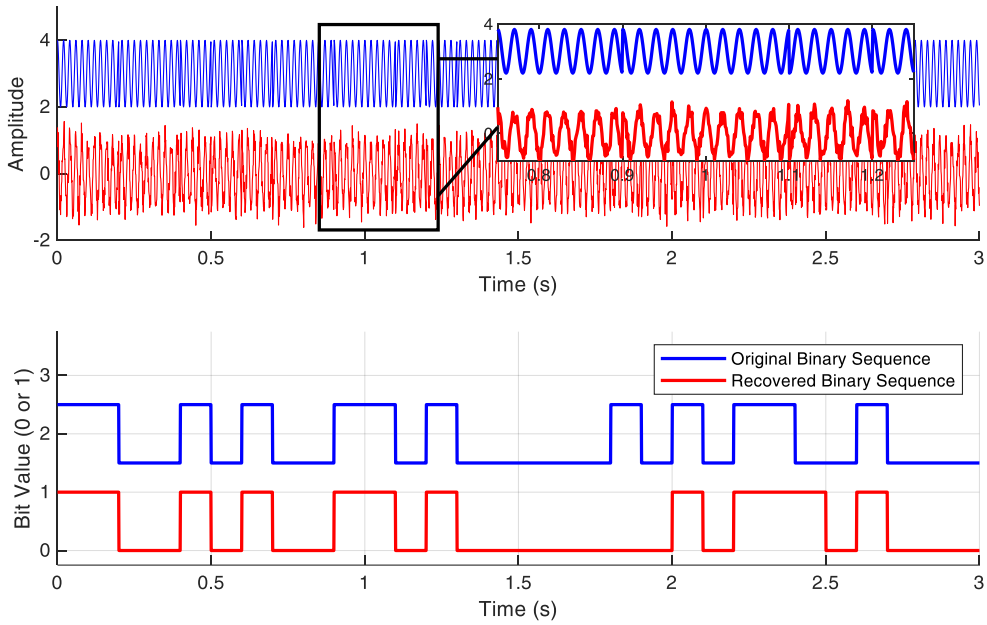
B) Simulation des effets du bruit de scintillation sur un signal numérique

Objectif : Etudier l'effet du bruit de scintillement dans le cas d'une modulation de phase

1. Générer une porteuse de 50Hz, d'une durée totale de 2s, d'amplitude égale à 1 et d'une amplitude de 0,5. Puis l'altérer avec un bruit de scintillation d'une variance de 30%. Les représenter dans le domaine temporel et également dans le domaine spectral.
2. Générer une porteuse échantillonnée à Fs=1kHz, contenant 30 symboles binaires durant chacun 10ms. Puis la moduler en phase avec une séquence binaire arbitraire (ou aléatoire) avant de l'affecter d'un bruit blanc de manière à obtenir un SNR de 3dB. Représenter le signal d'émission ainsi que le signal de réception. Représenter également la trame source ainsi que la trame reconstituée.
3. Calculer le BER pour un SNR allant de 1 à 15dB puis représenter cette caractéristique pour plusieurs transmissions.

4. Refaire les deux précédentes questions avec cette fois un bruit de scintillation avec comme variable la variance du bruit de scintillation au lieu du SNR du bruit blanc.






```
% Parameters
fs = 1000;              % Sampling frequency (Hz)
T = 2;                  % Total duration (seconds)
f_carrier = 50;         % Carrier frequency (Hz)
A = 0.5;                  % Carrier amplitude

% fonctions utiles
% randn
% fftshift et fft
```

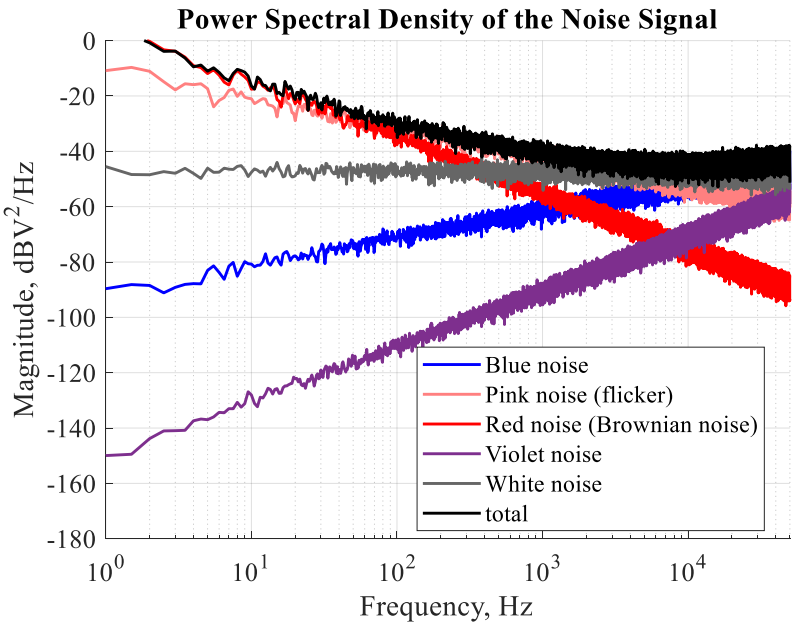
```
% Parameters
fs = 1000;                % Sampling frequency (Hz)
t_symbol = 0.1;           % Duration of one symbol (seconds)
n_symbols = 30;           % Number of symbols in the sequence
T = n_symbols * t_symbol; % Total signal duration (seconds)
f_carrier = 50;           % Carrier frequency (Hz)
A = 1;                    % Amplitude of the carrier

% fonctions utiles
```

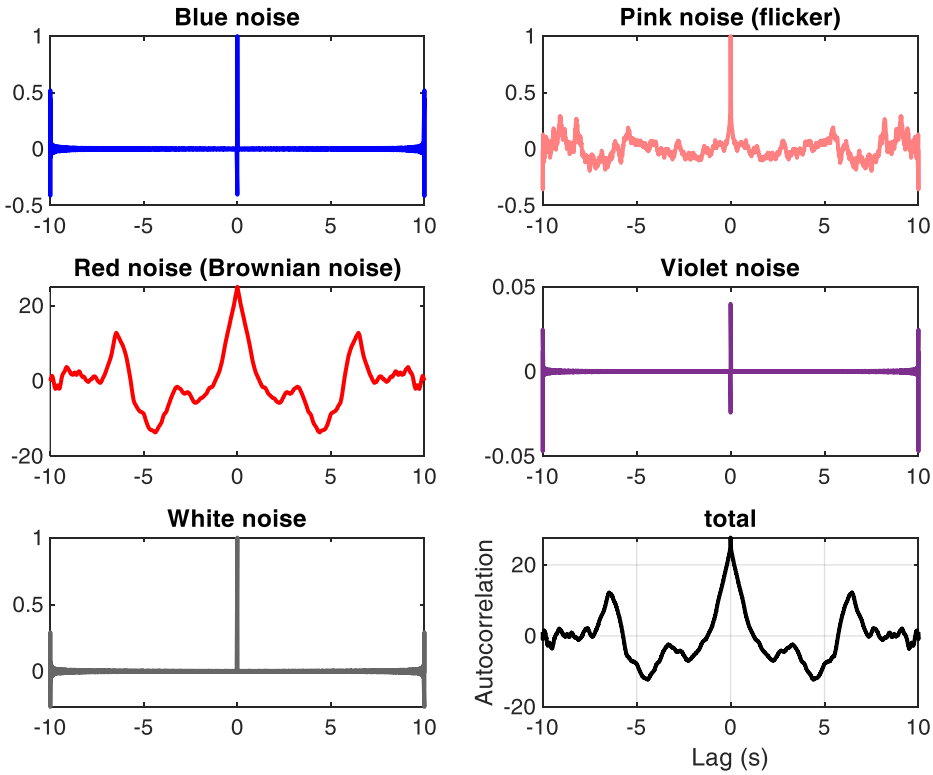
```
% randi
% awgn
% butter(5, 01, 'low') et filtfilt
% awgn
```

C) Analyse du bruit par autocorrélation

<u>Objectif :</u> Etudier l'intérêt de l'outil qu'est l'autocorrélation pour l'identification de différents bruits

1. A l'aide des fonctions de génération des bruits « colorés » fournis, générer des séquences d'un Méga échantillons de ces différents bruits puis les représenter avec les couleurs qu'on leur associe.
2. Calculer et représenter l'autocorrélation de chacun de ces bruits. Que dire de la capacité de cet outil à identifier les bruits.



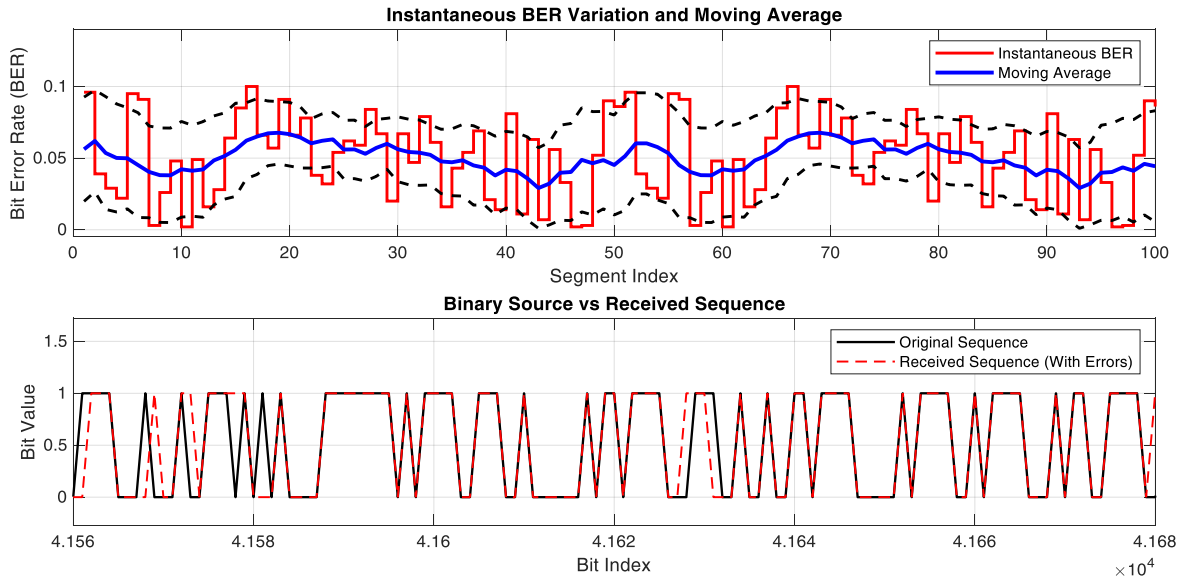


```
clear, clc

% signal parameters
fs = 1e5;          % sampling frequency, Hz
T = 10;              % signal duration, s
N = round(fs*T);     % number of samples

% fonctions utiles
% bluenoise
% pinknoise
% rednoise
% violetnoise
% randn
% hanning(2*fs, 'periodic');
% pwelch(x, window, noverlap, nfft, fs, 'onesided');
% log10
% semilogx(

nm={'Blue noise','Pink noise (flicker)', 'Red noise (Brownian noise)', 'Violet noise','White noise', 'total'};
clr=[0 0 1; 1 0.5 0.5; 1 0 0; 0.4940 0.1840 0.5560; 0.4 0.4 0.4;0 0 0];
```

D) Variance du taux d'erreurs binaires BER

<u>Objectif :</u> Caractériser statistiquement le taux d'erreurs binaires BER d'une transmission numérique.

1. Générer une séquence binaire de 100k échantillons aléatoire puis introduire des erreurs binaires à raison de 50 différents taux d'erreurs répartis sur les 100 segments binaires qui compartimente la séquence binaire.
2. Représenter en rouge le dérouler des différents taux d'erreurs sur l'ensemble des 100 segments.
3. Calculer puis représenter en bleu et en pointillé noir, la moyenne et la variance BER, respectivement. Cela, avec une fenêtre glissante de 100 échantillons.




```
clc; clear; close all;

%% Parameters
N = 100000; % Total number of bits
segment_size = 1000; % Segment length for BER calculation
num_segments = N / segment_size; % Number of segments

%% fonctions utiles
% randi
% window_size = 10;
% movmean et movvar
```